Compito 9.2

Malchiodi Riccardo 5680500

Obiettivo del Laboratorio

l'obiettivo di questo laboratorio è quello di implementare l'algoritmo *MCStringEqualityVerifier*. L'idea alla base dell'algoritmo è usare l'aritmetica modulare per calcolare una impronta digitale (fingerprint). Invece di confrontare a e b bit per bit, confrontiamo il resto della divisione di questi numeri per un numero primo p scelto a caso. Definiamo l'impronta digitale $f(n)$ come:

$$f(n) \ = \ n \ (mod \ p)$$

Questo approccio permette di verificare l'uguaglianza in modo efficiente, accettando una minima probabilità di errore (falsi positivi) che diminuisce ripetendo il test con numeri primi diversi (scelti ovviamente in modo casuale).

Analisi Output

```
stringa a: 11011100110101101101 (valore decimale: 904557)
stringa b: 11011100110101101100 (valore decimale: 904556)
Totale di iterazioni: 20
Totale volte 'input diversi' (Corretto): 20
Totale volte input probabilmente uguali' (Falsi Positivi): 0
Percentuale di errore in questo test: 0.0%
```

Ho deciso di scegliere, non in modo casuale, due stringhe che differiscono solo di un bit. L'output ottenuto mostra che su **20 iterazioni**, l'algoritmo ha restituito **20 volte "input diversi"** e **0 volte "input probabilmente uguali"**, con una percentuale di errore dello **0%**.

Interpretazione Matematica

Il cuore dell'algoritmo è il confronto tra le impronte digitali calcolate:

$$f(a) \ = \ a \ (mod \ p) \ \text{ e } f(b) \ = \ b \ (mod \ p)$$

Un **falso positivo** si verificherebbe se le impronte risultassero identiche

$f(a) = f(b)$ pur essendo le stringhe di partenza diverse $a \; != \; b$.

In questo esperimento ho scelto due stringhe tali che la loro differenza numerica è:

$$a \; - \; b \; = \; 1$$

Analizziamo perché l'errore è stato nullo:

1. La condizione di collisione $f(a) = f(b)$ equivale alla congruenza $a \equiv b \; (mod \; p)$
2. Per definizione di congruenza modulare, questo avviene se e solo se p è un divisore della differenza $(a \; - \; b)$.
3. Sostituendo i nostri valori, avremmo un falso positivo solo se il numero primo p dividesse **1.**
4. Poiché nessun numero primo $p \geq 2$ può dividere 1, la condizione $f(a) = f(b)$ non può mai verificarsi per questa specifica coppia di stringhe.

l'assenza di falsi positivi in questo test non è casuale, ma è strutturale: per questi specifici input, è matematicamente impossibile che le impronte $f(a)$ e $f(b)$ coincidano, indipendentemente dal primo p scelto.

Esperimento 2: Stringhe casuali

Per verificare il comportamento dell'algoritmo in uno scenario più realistico e meno controllato, ho eseguito un secondo test generando casualmente due interi a 20 bit distinti.

Analisi Output

```
--- ESPERIMENTO 2: STRINGHE CASUALI ---
Random A: 306455 (bin: 1001010110100010111)
Random B: 943520 (bin: 11100110010110100000)
Differenza |A - B|: 637065
Collisione trovata con p=11
Collisione trovata con p=13
Totale iterazioni: 20
Falsi Positivi su input casuali: 2
Percentuale di errore empirica (Random): 10.0%

--- ANALISI TEORICA ---
Lunghezza stringhe (l): 20 bit
Limite teorico di errore (2*ln(l)/l): 0.2996 (circa 29.96%)
```

A differenza del primo esperimento, in questo caso l'algoritmo non è stato infallibile. Su 20 iterazioni, abbiamo ottenuto:

- **18 volte "Diversi"** (Corretto)
- **2 volte "Probabilmente uguali"** (Falso Positivo)
- **Percentuale di errore:** 10.0%

Interpretazione Matematica dell'Errore L'output ci segnala esplicitamente che la collisione è avvenuta con il numero primo $p = 11$ e con $p = 13$. Analizziamo matematicamente perché è successo:

1. La differenza tra i due numeri generati è 637065.
2. Osserviamo che 637065 è un numero **dispari**.
3. Di conseguenza, la differenza è divisibile esattamente per 11 e anche per 13:

$$637065 \equiv 0(mod\ 11)$$

$$637065 \equiv 0(mod\ 13)$$

4. Poiché il numero 11 e il numero 13 sono presenti nella nostra lista di numeri primi candidati, quando l'algoritmo ha estratto casualmente $p = 11$ e $p = 13$, si è verificata la condizione di collisione:

$$A \equiv B(mod\ 11)$$
$$A \equiv B(mod\ 13)$$

Infatti, entrambi i numeri sono dispari, quindi entrambi danno resto 0 se divisi per 11 e 13 (0=0).

Questo risultato conferma la natura probabilistica dell'algoritmo Monte Carlo: se la differenza tra i file è un multiplo di uno dei primi nel nostro insieme di campionamento, esiste una probabilità non nulla (in questo caso 2/20) di ottenere un falso positivo.

Conclusioni

L'implementazione e il test dell'algoritmo *MCStringEqualityVerifier* hanno permesso di evidenziare due comportamenti distinti che caratterizzano gli algoritmi probabilistici:

1. **Caso Deterministico (Best Case):** Nel primo esperimento, la differenza tra i dati (1) non aveva divisori comuni con l'insieme dei primi scelti. In questo scenario, l'algoritmo è infallibile (errore 0%).
2. **Caso Probabilistico (Average Case):** Nel secondo esperimento con dati casuali, la differenza (637065) era divisibile per due dei numeri primi disponibili (11 e 13). Questo ha portato alla generazione di due falsi positivi..

Sintesi finale L'esperimento dimostra che il *fingerprinting* modulare è estremamente efficiente in termini di calcolo, ma non garantisce la correttezza assoluta in una singola esecuzione. Tuttavia, l'errore osservato (10%) è basso e può essere ulteriormente ridotto aumentando il numero di iterazioni (runs) o ampliando l'intervallo dei numeri primi utilizzati.

```
import random

import math



# funzione per trovare tutti i numeri primi tra 2 e 400

def prime_numbers_list(limite):

   prime_numbers = [2]

   num = 3

   while num < limite:
```

```
        for i in range (2,num):
            if num % i == 0:
                break
            elif i == num-1:
                prime_numbers.append(num)
        num += 1
    return prime_numbers


# funzione per pescare p tra 2 e 400
def get_random_p(prime_list):
    p = random.choice(prime_list)
    return p


def MCStringEqualityVerifier(a, b, lista_primi):
    p = get_random_p(lista_primi)


    # calcolo impronta digitale di a e b
    f_a = a % p
    f_b = b % p


    # verifica se le impronte sono uguali
    if f_a == f_b:
        return "input probabilmente uguali", p
    else:
        return "input diversi", p
```

```python
l = 20

limite = l**2


# Generiamo la lista dei primi una volta per tutte

lista_primi = prime_numbers_list(limite)


bin_a = "11011100110101101101"

bin_b = "11011100110101101100"


int_a = int(bin_a, 2)

int_b = int(bin_b, 2)

print(f"stringa a: {bin_a} (valore decimale: {int_a})")

print(f"stringa b: {bin_b} (valore decimale: {int_b})")


falsi_positivi = 0

runs = 20


for i in range(runs):

  messaggio, p_estratto = MCStringEqualityVerifier(int_a, int_b, lista_primi)


  if messaggio == "input probabilmente uguali":

        falsi_positivi += 1


# --- RIEPILOGO FINALE ESPERIMENTO 1 ---

print(f"Totale di iterazioni: {runs}")

print(f"Totale volte 'input diversi' (Corretto): {runs - falsi_positivi}")
```

```
print(f"Totale volte 'input probabilmente uguali' (Falsi Positivi):
{falsi_positivi}")


perc_errore = (falsi_positivi / runs) * 100

print(f"Percentuale di errore in questo test: {perc_errore}%")


print("\n--- ESPERIMENTO 2: STRINGHE CASUALI ---")


# Generiamo due numeri casuali di 20 bit

# Assicuriamoci che siano diversi per testare i falsi positivi

int_a_rand = random.getrandbits(20)

int_b_rand = random.getrandbits(20)

while int_a_rand == int_b_rand:

   int_b_rand = random.getrandbits(20)


print(f"Random A: {int_a_rand} (bin: {bin(int_a_rand)[2:]})")

print(f"Random B: {int_b_rand} (bin: {bin(int_b_rand)[2:]})")


diff_rand = abs(int_a_rand - int_b_rand)

print(f"Differenza |A - B|: {diff_rand}")


falsi_positivi_rand = 0


for i in range(runs):

   risultato, p = MCStringEqualityVerifier(int_a_rand, int_b_rand, lista_primi)
```

```
    # Se dice "Probabilmente uguali" ma i numeri sono diversi, è un falso positivo
    if risultato == "input probabilmente uguali":
        print(f"Collisione trovata con p={p}") # Vediamo quale primo ha causato
l'errore
        falsi_positivi_rand += 1




print(f"Totale iterazioni: {runs}")
print(f"Falsi Positivi su input casuali: {falsi_positivi_rand}")
perc_errore_rand = (falsi_positivi_rand / runs) * 100
print(f"Percentuale di errore empirica (Random): {perc_errore_rand}%")


# --- CONFRONTO CON IL LIMITE TEORICO ---
# Calcolo della probabilità teorica secondo la proposizione 9.3 delle dispense
prob_teorica = (2 * math.log(l)) / l
perc_teorica = prob_teorica * 100


print("\n--- ANALISI TEORICA ---")
print(f"Lunghezza stringhe (l): {l} bit")
print(f"Limite teorico di errore (2*ln(l)/l): {prob_teorica:.4f} (circa
{perc_teorica:.2f}%)")


if perc_errore_rand <= perc_teorica:
    print(f"CONCLUSIONE: L'errore empirico ({perc_errore_rand}%) rientra nel limite
teorico previsto.")
else:
```

```
    print(f"NOTA: L'errore empirico è superiore alla media teorica (può capitare su
pochi run).")
```